

Team Information

Name: Chess++

Group Members:

- Ahmad Bilal Carini
 - Position: Version Control Management
- Zeferino Araiza-Flores
 - Position: Timeline Management
- Zenas Ortega
 - Position: Chess Management

Communication Tools:

- Microsoft Teams
- School Email

Communication Rules:

- Respond 24 hours from message
- Ping everyone for important messages
- If no response then an email will be sent

Relevant Project Artifacts:

https://github.com/ZeferinoA/Chess_plusplus

Product Information

Abstract:

- This program can simulate a chess game for beginners and advanced players who want to take the game into their own hands. Beginners will be able to learn the basics of how each piece moves up to the complex openings and book moves. Advanced players will be able to improve the game that they have come to learn with pieces they want to add and establish their moves. These players can then play against another or a Stockfish CPU Agent

Goal:

- The goal is to teach users how to play chess, learn FEN notation, and new ways to play, and allow for local 2-player and Stockfish CPU games

Current practice:

- Chess exists, but can a chess board teach new chess players how to play? How about with new pieces that can do dynamic and interesting moves? What about a simple CLI chess interface that supports complex moves like Castling and En

Passants? Plenty of CLI chess games exist, but do they have the wealth of features that truly makes them worth using over a physical chess board?

Novelty:

- The novelty of this program will be the CLI chess interface that supports complex moves, the addition and use of new pieces and new ways to play the game, and local connectivity.

Do not reinvent the wheel or reimplement something that already exists, unless your approach is different.

Effects:

- New players that want to learn the basics and the complexities of the game will have a place to learn. Players that want a unique twist to the game can create new pieces to use and go against. It will allow all players to have fun either by learning to play for the first time, good ol simple chess, or have crazy modes/new pieces implemented

Technical Approach:

- The program will run in a linux environment and may support other OS and platforms, it will be developed primarily in C++, leveraging the use of rendering libraries for the graphical elements. It will be developed and tested through a command line interface. Git and Github will be used for planning and version control. Pull requests will be used to manage merging.

Risks:

- Creating an intuitive interface for moving pieces could pose a problem. Chess notations exist but require outside knowledge beyond just chess knowledge. Overall, the interface poses a risk as it may end up being too esoteric to understand how the program works if the interface ends up being too complex or limiting.
- Maintaining the novelty of the program despite the ubiquity of chess could pose a challenge. Making novel versions and modes or pieces could add flavor to the project but making sure the game remains balanced with those new pieces or versions will pose a challenge.

- When working through Github, making sure merges don't break or overwrite features that others have created will pose a risk, even though Github allows easy access to rollbacks.'
- Programming lesser known features of chess, such as En Passants, may pose an issue and leaving these features out (because of knowledge limitations or negligence) would remove a core (if obscure) component of the game.

Use Cases (Functional Requirements): Each team member must come up with and describe **at least one** use case of your product. Refer to "1. Use Cases (Functional Requirements)" in [Project Requirements Elicitation](#)

1. Learn and Practice Chess, Beginner Level

- Actors: User that is a beginner to chess and wants to learn
- Triggers: A user selects the lesson option, and begins their level of beginner
- Preconditions: User has opened the game, loads on their system, and the standard rule set is embedded in the system
- Postconditions (success scenario): The user is able to start the basic lessons in which the program teaches about the pieces and basics
- List of steps (success scenario)
 - User selects the lesson menu
 - User selects the basic lesson they want to study
 - The system bring up a text line that states what is needed to do with explanation
 - User selects the right move
 - The lesson is complete and the user can now move on to the next lesson
- Extensions/variations of the success scenario: The variation/extension of this can be the one used for the advanced setting
- Exceptions: A user puts in an invalid input and is told to insert in a certain format. A user selects the wrong move and is told that it is not correct and is retold what is needed

2. Learn and Practice Chess, Advanced Level

- Actors: User that is decently experienced with chess and wants to learn more
- Triggers: A user selects the lesson option, and begins their level of advanced

- c. Preconditions: User has opened the game, loads on their system, and the standard rule set is embedded in the system
 - d. Postconditions (success scenario): The user is able to start the advanced lessons in which the program teaches about book moves and openings
 - e. List of steps (success scenario):
 - i. User selects the lesson menu
 - ii. User selects the advance lesson they want to study
 - iii. The system bring up a text line that states what is needed to do with explanation
 - iv. User selects the right move
 - v. The lesson is complete and the user can now move on to the next lesson
 - f. Extensions/variations of the success scenario: The variation/extension of this can be the one used for the basic setting
 - g. Exceptions: A user puts in an invalid input and is told to insert in a certain format. A user selects the wrong move and is told that it is not correct and is retold what is needed
3. Play against CPU through Stockfish-CLI bridge while being able to analyze moves in realtime
- a. Actors: User that wishes to learn chess in real games and play against a CPU opponent
 - b. Triggers: Loads a board linked and updated to a Stockfish agent running, allowing the user to play against a CPU, another person, or analyze positions dynamically,
 - c. Preconditions: User has opened the game and it loads on their system, Stockfish executable is in the proper subdirectory
 - d. Postconditions: The chess bridge loads and properly supplies the current FEN notation to stockfish to initialize a board. Menu displays options and analytical components load across bridge
 - e. List of steps (success scenario):
 - i. Player selects difficulty option in main menu
 - ii. Player selects CPU difficulty from prompted options
 - iii. Player selects Play v. CPU option
 - iv. Standard board config (Chess default) loads and board displays
 - v. Player selects to “list valid moves”
 - vi. Player selects “make a move”
 - vii. Player selects from valid moves for “make a move”

- viii. Board updates
 - ix. Stockfish automatically begins thinking process and makes move
 - x. Player repeats until losing then selects “analyze position”
 - xi. Using analysis, the player learns from their position and becomes better at chess.
- f. Extensions/variations of the success scenario
 - i. Player selects “find best move” option in order to get out of bad position in order to keep playing through the game
- g. Exceptions: failure conditions and scenarios
 - i. The pieces do not save
 - ii. Stockfish fails to connect
 - iii. FEN notation (input) does not properly send to Stockfish agent

Non-functional Requirements: Describe at least three non-functional requirements, e.g., related to scalability, usability, security and privacy, etc.

- Scalability
 - The scalability of the Stockfish CPU is reliant on local CPU speed for thinking speed, especially on low difficulties where thinking time is severely limited
- Adaptability
 - The application needs to adapt to the users skill level for CPU games and Lessons
- Usability
 - Since the application is a game, the user experience needs to be friendly and the lessons given by the application needs to be understandable for the user.

External Requirements: Refer to "3. External Requirements" in [Project Requirements Elicitation](#).

- As our application will take user input for moves and chess piece creations we do need to check the external requirements with being robust against errors
- Since the app is in c++, the application will be downloaded and be able to be executed by the user

Major Project Features:

- Major Feature 1: Establishing the logic of the command-line chess, beginning with the creation of the array for the board, game pieces structures, and the rules for the game
- Major Feature 2: Establishing the logic for the chess teacher for basic moves such as how the pieces move and how to check and checkmate
- Major Feature 3: Expanding upon the learning logic for complex move such as special openings and book moves such as castling
- Major Feature 4: Allowing users to create their own chess pieces with their own move set in order to expand the game

Stretch Features:

- Stretch goal 1: Making a local multiplayer environment so that the game is playable between players on the same network but different devices through the use of network sockets and hosting clients (as needed).
- Stretch goal 2: Making Stockfish Post Analysis Bridge for Player vs. Player local games. Stockfish supplies Analysis features, but adding analysis for local PVP games would require the bridge to be integrated even in normal games which is another layer of programming integration.

Team process description

Describe your quarter-long development process.

- Specify and justify the software toolset you will use.
 - We chose to use C++ since it is high-level language that can seamlessly integrate with Stockfish and is a comfortable language with our team with all of us knowing how to do it and it is a real challenge with the less libraries
- Define and justify each team member's role: why does your team need this role filled, and why is a specific team member suited for this role?
 - Our team needs the roles of Version Control Management, Timeline Management, and Chess Management due to the method of the project being tight and compact with the deadline and the necessity of Chess being implemented in our Chess app. The Version Control Manager is needed for the repository and making sure that no code is pushed without proper checks, Ahmad is a great fit for this role for their great availability in

checking code, being a leader and mentioning when something needs to be revisited, and they're avid about maintaining good working code. The Timeline Manager is needed for reaching the deadlines and week schedules that we have for the project, Zeferino is a great fit for this role for the schedule checking with history with managing the project tab of a previous project and the ability to work with a deadline. The basics of the app needed is the chess system so a Chess Manager is needed to make sure the project remains to the core principle, Zenas is needed for this role for his knowledge in chess and the passion he had carried for pitching this project to the team.

- Provide a schedule for each member (or sub-group) with a measurable, concrete milestone for each week. "Feature 90% done" is not measurable, but "use case 1 works, without any error handling" is.
 - Week 4: The project has begun and the folder directory of github is specified and version control measures have been implemented for all future commits.
 - Week 5: We have a completed the set of elicited scenarios for the beginner level (use-case 1) and advanced level (use-case 2)
 - Week 6: Chess++ is a functional game in the command line, that is, you can play a game of chess against a friend.
 - Week 7: Use-case 1 scenarios are implemented in the game and Use-case 2 scenarios are implemented in the game.
 - Week 8: Use-case 3 is complete, without error handling
 - Week 9: Stretch goals have been evaluated for possibility and those that were deemed possible were completed.
 - Week 10: The project now has error-handling and UI/UX has been evaluated to ensure the game is in a playable state.
- Specify and explain at least three major risks that could prevent you from completing your project.
 - The implementation of Use Case 3 proves too challenging to implement well. That is: the implementation of Stockfish proves too challenging to run locally. This is because of the team's inexperience running local machine learning agents that can deliver unexpected behaviors.

- The implementation of guides and learning systems requires technical knowledge of chess that no one in the group has. The general wide range of skill gaps that chess as a game fosters could lead advanced players to not feel like their skills are truly explained well for advanced scenarios.
- None of the stretch goals specified are feasible within the 10 week time span of the course and we end up only being proficient enough to implement the basic requirements.
- Describe at what point in your process external feedback (i.e., feedback from outside your project team, including the project manager) will be most useful and how you will get that feedback.
 - The point that external feedback will be processed will be at the start, after the presentation and comments, since this can challenge us to add things that are wanted by the feedback or things to make different that we initially pitched. The next time to gather feedback would be after the creation of our lessons, we want to know what lessons need or are wanted to be included in the application since time is limited for the project and we want the most impact for the userbase. The final time to gather feedback would be after implementation of the creation feature as we want to know about how we can make the creation better for the users or what to add on such as which stretch goal is wanted to be implemented first or if anything past that is wanted.

Timeline:

- **Week 1:** Begin creating array chess board with pieces
- **Week 2:** Begin the logic of the chess board for check, checkmate, and others
- **Week 3:** Create the teaching module for basic moves such as how the pieces move
- **Week 4:** Expand the teaching module for basic moves such as checking and checkmating
- **Week 5:** Expand the teaching module for complex moves with openings
- **Week 6:** Expand the teaching module for complex moves with castling and other strategies
- **Week 7:** Allow for user creation of pieces and rules for them
- **Week 8:** Allow for the use of the pieces to be integrated into the board
- **Week 9:** Attempt stretch goals in order to allow playability with these pieces

Software architecture:

Provide an overview of your system. Specifically:

- Identify and describe the major software components and their functionality at a conceptual level.
 - **Chess Engine Core:** The core manages the board array, piece structures, and standard move validation.
 - **Instructional Plug in:** A teaching system that guides users through beginner lessons and advanced strategies.
 - **Stockfish agent Plug-in:** Stockfish agent that can play and analyze Chess while supporting a full moveset.
- Specify the interfaces between components.
 - **CLI to Engine:** The primary interface where user-inputted chess notation or menu selections are parsed and passed to the game logic.
 - **Engine to Plug Ins:** The engine will connect the command line interface to the area that it needs to go to.
- Describe in detail what data your system stores, and how. If it uses a database, give the high level database schema. If not, describe how you are storing the data and its organization.
 - **In-Memory Storage:** Current game states and custom pieces are stored in memory for the duration of the session.
 - **No Database:** Standard C++ data structures will be used to organize the board and pieces.
- If there are particular assumptions underpinning your chosen architecture (Microkernel), identify and describe them.
 - **Consistency:** We assume all plug-ins can work with the same coordinate-based input system and can be properly parsed
- For each of two decisions pertaining to your software architecture, identify and briefly describe an alternative. For each of the two alternatives, discuss its pros and cons compared to your choice.
 -

Alternatives	Pros	Cons
Event-Driven	The real-time processing will be able to respond to events quicker than the plug-in	Testing will be more complicated due to the timing of events compared to code
Layered Architecture Pattern	Flexibility as the plug-ins are not independant	Due to the layers not communicating, more will have to be done to make a working chess app

Software design:

Component 1: Chess Engine Core

- **Parts:**
 - **ChessKernel Class:** The main game loop that will call the other functions needed to run the program
 - **Board Class:** An array, vector-based, or other C++ structure grid representing the game
 - **RuleValidator:** The logic that will establish the basics of chess such as check, moves, etc
- **Responsibilities:**
 - Handling basic CLI input and output
 - Establishing the central logic of the game
 - Handling basic chess operations

Component 3: Customization Plug in

- **Parts:**
 - **Customization Interface:** The creation base for user pieces in which moves and symbol will be selected
 - **CustomStorage:** A container that stores user's custom pieces
- **Responsibilities:**
 - Defining move patterns.
 - Assigning custom characters icons for pieces
 - Storage for pieces in order for player to use

Component 2: Instructional Plug-in

- **Parts:**
 - **LessonManager Class:** A class to maintain the storage of the lessons and that can control progression
 - **Lesson Struct:** Specific board setups and goal for the user to complete
- **Responsibilities:**
 - Providing feedback to the user based on their move
 - Managing user progression and storing it

Coding guideline:

- **Language:** C++.
- **Guideline:** [C++ Core Guidelines](#)

- **Rationale:** We chose the C++ Core Guidelines due to its popularity with the programming community. There is a google one but it is seen as more legacy and we want the most up to date and living of the documentation.
- **Enforcement:** The Version Control Manager will enforce these through mandatory code reviews on Pull Requests. Revisions will be made if not followed

Process description:

1. Risk assessment

Risk	Likelihood	Impact	Evidence	Reduction Steps	Detection Plan	Mitigation
Stockfish Bridge Complexity	High	High	Stockfish is an unknown program as far as current team experience extends.	Begin by outlining robust bridge class capable of running a devised protocol that integrates with the Stockfish executable	Unit tests on "Move Validator" using custom pieces.	Plug-in functions for additional functionality that don't break other code (analysis functions, best move functions, etc.)
Chess Notation Assistance	Medium	High	Chess notation is hard for beginners.	Implement a helper that displays valid moves with grid.	Usability testing with non-chess players.	Include a tutorial or help menu for chess notation.

Chess Knowledge Gap	Medium	Medium	The team lacks advanced chess expertise.	Seek out the usual advance moves that are taught to basic players	Peer review of lesson content by the Chess Manager.	Focus the "Advanced" lessons on basic strategy rather than chess theory.
Merge Conflicts/Broken Builds	Medium	High	Frequent updates to the central engine will cause conflicts.	Use a strict Microkernel interface so plug-in code stays separate from core code.	GitHub Actions to run automated builds on every Pull Request.	Revert to the last stable commit using GitHub's rollback.

2. Project schedule

Milestone	Tasks	Effort Est.	Dependencies
M1: Core Engine Functional	1. Implement Board 2. Code Standard Move Logic	2 Person-Weeks	None
M2: Lesson Plug-in	1. Implement Tutorial Menu	3 Person-Weeks	M1 Complete

	2. Create Beginner Lessons. 3. Create Advanced Lessons		
M3: Stockfish and translation	1. Create Custom Piece Interface 2. Build Creation Menu	2 Person-Weeks	M1 Complete
M4: System Integration	1. Integrate all Plug-ins 2. Implement Error Handling	1 Person-Week	M2, M3 Complete

3. Team structure

- **Ahmad (Version Control Manager):** Responsible for repository, Pull Request approvals, and maintaining coding guidelines.
- **Zeferino (Timeline Manager):** Responsible for tracking weekly milestones and ensuring features are delivered per the schedule.
- **Zenas (Chess Manager):** Responsible for the accuracy of chess logic, tutorial content, and the design of custom piece mechanics

4. Test plan & bugs

- **Integration Testing:** We will test the interaction between the Kernel and Plug-ins, ensuring that a custom piece does not corrupt the standard board array.
- **Usability Testing:** We will conduct "blind tests" where a user attempts to complete a beginner lesson without prior instructions to ensure the CLI is intuitive.

5. Automated Testing and CI

- **Automated Testing:**

- **Framework Choice:** Catch2 will be the one we use for automated testing framework
- **Justification:**
 - Lightweight so we don't have to add anything for our offline terminal based app
 - Automatically makes a main function that can be integrated easily with our makefile
 - The test structure is very simple so calls for it in the terminal make it all the easier
- **Adding Test:** Adding test is simple with simply adding a new test with the title format of "test_case_#" to the /tests directory in the repo. A test is written in the usual Catch2 format with header and test. Makefile already takes care of test file build so the command "make run-tests" is the only thing needed to do for compilation and execution.

- **CI:**

- **Choice and How it is Linked:** Github Actions, this is linked to the repo by adding a .github directory with a ci.yml file that is automatically read and ran by Github
- **Justification:**
 - The repository is already on Github so addition is very easy so no external service is required
 - It supports the linux environment that we are using
 - Automatically work with our push and pull request
- **Pros/Cons:**
 - **Pros**
 - Easily connected and executed to the repo
 - Automatic test ran and enforced
 - Can be scalable for all the test that are added and any dependencies needed
 - **Cons**
 - Resource limits if our app does requires more memory or processing
 - Limited debugging information given after the execution
 - Since it is on the cloud, the build time takes a longer while and reduces our offline aspect

- **Test to be Executed:**
 - Piece movement validation
 - Illegal move rejection
 - Board state correctness
 - Check detection logic
 - ChessEngine interacting with RuleValidator
 - Game state after multiple sequential moves
 - Custom piece functionality integration
 - Beginner lesson move success
 - Advanced lesson progression
 - Custom piece creation and usage
- **Development:**
 - Code is pushed to main
 - A pull request is created
 - A pull request is updated

6. Documentation plan

- **User Guide:** A README on GitHub explaining how to launch the program.
- **Help Menus:** A help command that can be called by the user that will prompt a menu for commands to use.